

Open in app ↗

 Search Write

Stochastic Volatility Models Simulation and Statistical Model



Simon Leung · 14 min read · Dec 25, 2024



Mean Reversion Models (Ornstein-Uhlenbeck Process, Heston Model) and Volatility Clustering (GJR-GARCH Model)

Note to the reader: The purpose of writing this article is to help me gain knowledge, understand, and remember the material. If any reader finds any misinformation or errors, please do not hesitate to respond with corrections.

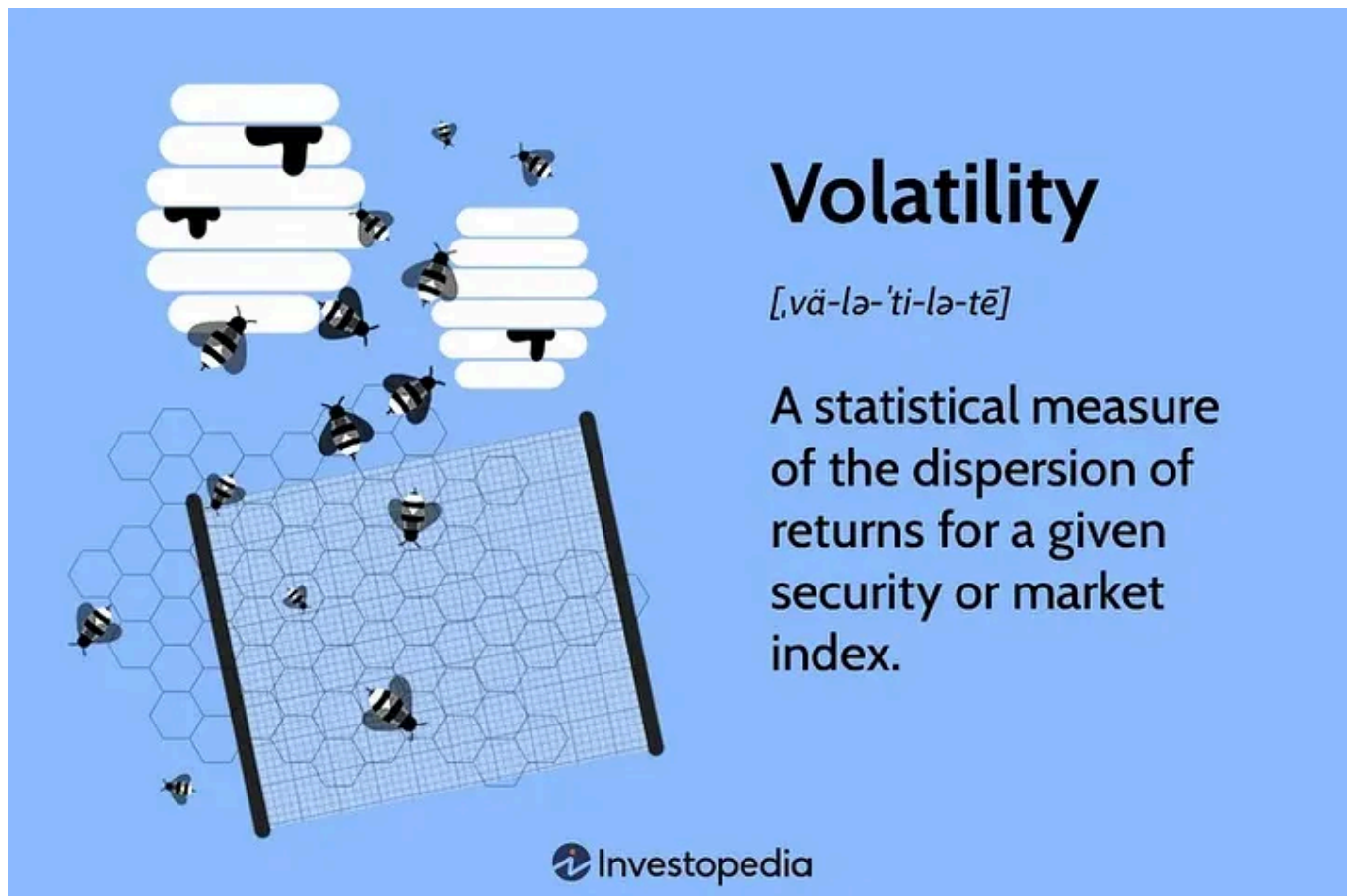


Image from <https://www.investopedia.com/terms/v/volatility.asp>

Financial time series exhibit several distinctive stochastic properties that make them particularly challenging to model and predict. ***Volatility clustering*** manifests as periods where market turbulence tends to persist, with large price changes typically followed by other large changes, creating distinct regimes of high volatility. ***Mean reversion*** describes the tendency of asset prices to gradually return to their long-term average, though the speed and strength of this phenomenon varies across different markets and timeframes. ***Heteroskedasticity***, a key feature of financial data, indicates that the variance of returns is not constant over time but rather changes dynamically, often in response to market conditions and new information flows. Perhaps most notably, financial returns typically display ***heavy-tailed distributions***, meaning that extreme events occur more frequently than would be expected under a normal distribution — this “fat tail” property has crucial implications for risk management, as it suggests that catastrophic

market moves are considerably more likely than standard models might predict. These interrelated properties create a complex statistical landscape that defies simple modeling approaches and necessitates sophisticated techniques for financial analysis and prediction.

Let load AAPL stock data:

```
import yfinance as yf

endDate = "2025-01-01"
startDate = "2020-01-01"

ticker = 'AAPL'

# Download AAPL stock data
aapl = yf.download(ticker, start=startDate, end=endDate)

# Calculate daily log returns
aapl['Log Return'] = np.log(aapl['Adj Close'] / aapl['Adj Close'].shift(1))

# Calculate daily volatility (standard deviation of log returns)
daily_volatility = aapl['Log Return'].std()

# Calculate 21-day rolling (historical) volatility
window = 21
aapl['21-Day Rolling Volatility'] = aapl['Log Return'].rolling(window=window).std()

# Calculate Annualize rolling volatility
trading_days = 252
#aapl['annualized_volatility'] = aapl['21-Day Rolling Volatility'] * np.sqrt(trading_days)
annualized_volatility = daily_volatility * np.sqrt(trading_days)

print(f"AAPL Stock daily Volatility: {daily_volatility:.2%}")
print(f"AAPL Stock Annualized Volatility: {annualized_volatility:.2%}")
```

```
AAPL Stock daily Volatility: 2.00%
AAPL Stock Annualized Volatility: 31.69%
```

Exhibit mean-reversion:

Use the Phillips-Perron (PP) test assesses whether a time series is stationary, a necessary condition for mean-reversion. Phillips-Perron (PP) test is robust to heteroscedasticity and autocorrelation.

If the p-value is low (typically less than 0.05), you reject the null hypothesis, indicating the series is stationary.

```
# https://pypi.org/project/arch/
#!pip install arch
from arch.unitroot import PhillipsPerron

# Apply the Phillips-Perron test
pp_test = PhillipsPerron(aapl['Log Return'].dropna())
print(f"Test Statistic: {pp_test.stat}")
print(f"P-Value: {pp_test.pvalue}")
print(f"Critical Values: {pp_test.critical_values}")

# Interpretation
if pp_test.pvalue < 0.05:
    print("Reject the null hypothesis: The series is stationary.")
else:
    print("Fail to reject the null hypothesis: The series is non-stationary.")
```

```
Test Statistic: -39.48969137761886
P-Value: 0.0
Critical Values: {'1%': -3.4355838322867998, '5%': -2.863851267837805, '10%': -2.575825314066143}
Reject the null hypothesis: The series is stationary.
```

Comparison with Other Tests:

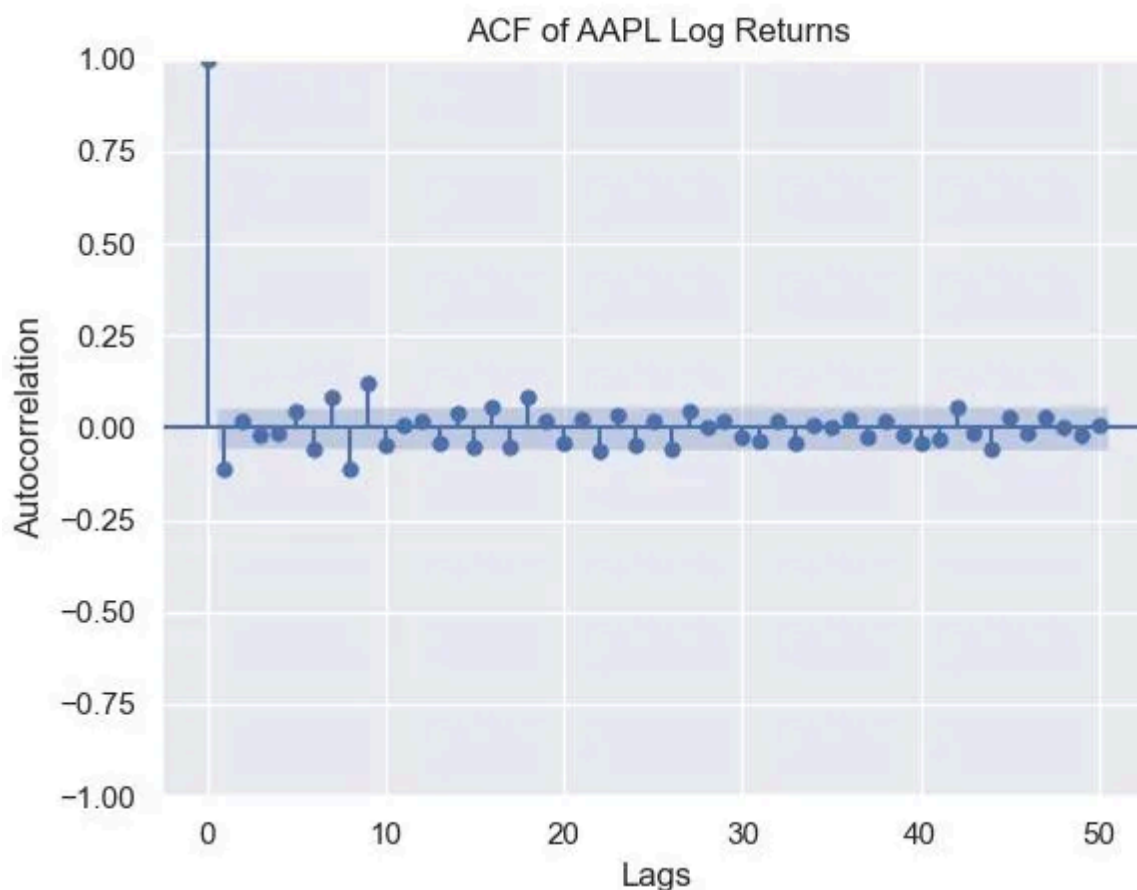
Test	Null Hypothesis	Strengths
ADF Test	Non-stationary	Simple and widely used.
KPSS Test	Stationary	Tests for trend or level stationarity.
Phillips-Perron	Non-stationary	Robust to heteroscedasticity and autocorrelation.

Another method: The Autocorrelation Function (ACF) measures the correlation between the time series and its lagged values. For mean-reversion, it will observe negative autocorrelation at certain lags, meaning that after a positive return, the subsequent returns tend to be negative (and vice versa).

```
from statsmodels.graphics.tsaplots import plot_acf

# Drop NaN values from the log returns
log_returns = aapl['Log Return'].dropna()

# Plot the ACF of the log returns
plt.figure(figsize=(10, 6))
plot_acf(log_returns, lags=30) # You can adjust the number of lags as needed
plt.title('ACF of AAPL Log Returns')
plt.xlabel('Lags')
plt.ylabel('Autocorrelation')
plt.show()
```



A significant negative ACF at lag 1 or at later lags (such as lag 5 or 7) would suggest that AAPL log returns are mean-reverting, as after a positive return, there is a tendency for negative returns to follow.

Also, the **Variance Ratio (VR) test** is a statistical method used to determine whether a time series follows a random walk or exhibits mean-reverting behavior.

```
# Drop NaN values from the log returns
log_returns = aapl['Log Return'].dropna()

# Function to calculate variance ratio for a given time horizon (h)
def variance_ratio(log_returns, h):
    N = len(log_returns)
    # Calculate the variance of the original series
    var_original = np.var(log_returns)
```

```

# Calculate the variance of the returns over lag h
log_returns_reshaped = [log_returns[i:i+h] for i in range(0, N-h)]
var_lagged = np.var([np.sum(block) for block in log_returns_reshaped]) / h

# Variance ratio: compare variance over time horizon h with original variance
return var_lagged / var_original

# Test the variance ratio for different time horizons (1-day, 5-day, 10-day, etc)
horizons = [1, 5, 10, 20, 50] # Different time horizons for the test
variance_ratios = [variance_ratio(log_returns, h) for h in horizons]

# Print out the variance ratios for interpretation
for h, vr in zip(horizons, variance_ratios):
    print(f"Variance Ratio for horizon {h} days: {vr}")

# Interpretation of the results:
for h, vr in zip(horizons, variance_ratios):
    if vr < 1:
        print(f"Horizon {h} days: Evidence of mean-reversion (VR < 1)")
    elif vr == 1:
        print(f"Horizon {h} days: Random walk behavior (VR ≈ 1)")
    else:
        print(f"Horizon {h} days: Evidence of persistence/trend-following (VR > 1)")

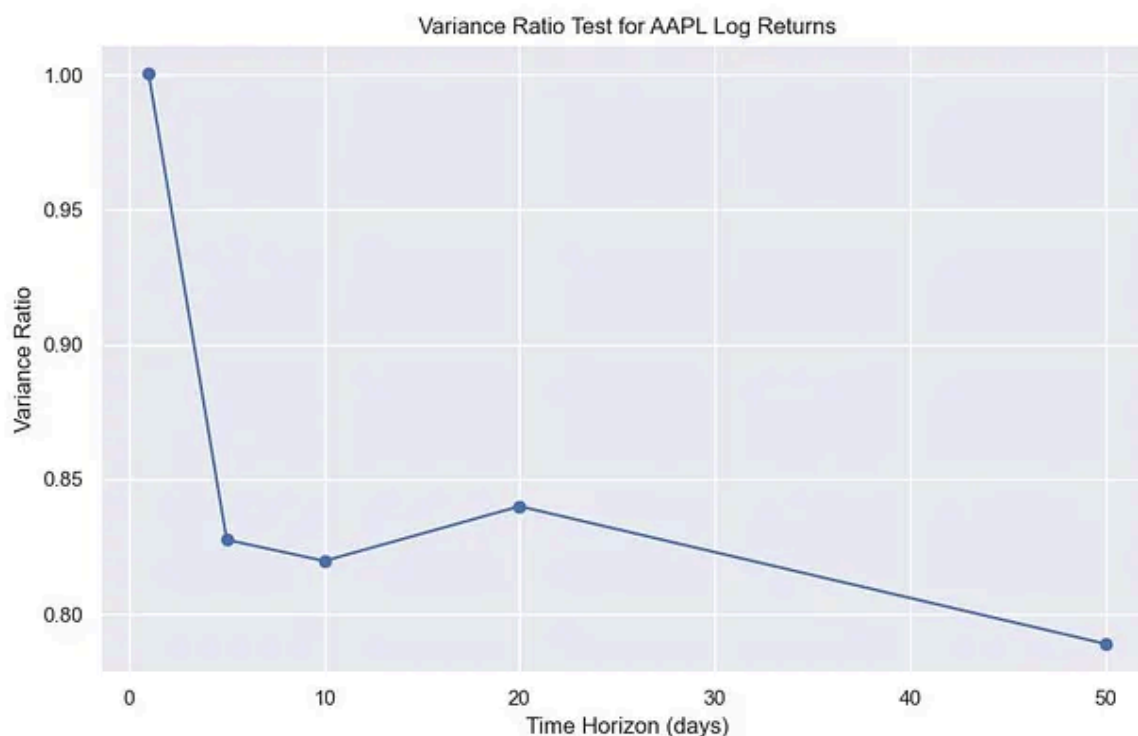
# Plot the variance ratios for visual inspection
plt.figure(figsize=(10, 6))
plt.plot(horizons, variance_ratios, marker='o')
plt.title('Variance Ratio Test for AAPL Log Returns')
plt.xlabel('Time Horizon (days)')
plt.ylabel('Variance Ratio')
plt.show()

```

Interpret the variance ratio:

- $VR < 1$: Suggests mean-reversion.
- $VR \approx 1$: Suggests a random walk (no mean-reversion).
- $VR > 1$: Suggests a persistent (trend-following) behavior.

Variance Ratio for horizon 1 days: 1.0006657001205692
Variance Ratio for horizon 5 days: 0.8275389087198917
Variance Ratio for horizon 10 days: 0.8196701933757288
Variance Ratio for horizon 20 days: 0.839957843857398
Variance Ratio for horizon 50 days: 0.7889154613041756
Horizon 1 days: Evidence of persistence/trend-following ($VR > 1$)
Horizon 5 days: Evidence of mean-reversion ($VR < 1$)
Horizon 10 days: Evidence of mean-reversion ($VR < 1$)
Horizon 20 days: Evidence of mean-reversion ($VR < 1$)
Horizon 50 days: Evidence of mean-reversion ($VR < 1$)



Modeling stock volatility with the Ornstein-Uhlenbeck (OU) process can be done by treating volatility as a mean-reverting process. The OU process is often used to model the evolution of volatility, where the process reverts toward a long-term mean. Using a method called Maximum Likelihood

Estimation (MLE) to estimate the parameters of the Ornstein-Uhlenbeck process. However, classic OU process is typically modeled as a Gaussian process with normally distributed increments. Instead of assuming normally distributed noise in the likelihood function, replace the normal log-likelihood in the MLE step with a Student's t-distribution log-likelihood.

Ornstein-Uhlenbeck Process

Definition:

The OU process is a continuous-time stochastic process described by the stochastic differential equation (SDE):

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t$$

where:

- μ : Long-term mean (mean-reversion level).
- θ : Speed of mean reversion.
- σ : Volatility (magnitude of randomness).
- W_t : Standard Brownian motion.

Characteristics:

- Models mean-reverting behavior around μ with strength controlled by θ .
- Gaussian distribution for X_t , leading to tractability in statistical analysis.

```
from scipy.optimize import minimize
from scipy.stats import t

log_returns = aapl['Log Return'].dropna().values

# Fit the Generalized OU Process with t-distributed increments
def ou_negative_log_likelihood(params, log_returns, dt):
    theta, kappa, sigma, nu = params
    n = len(log_returns)
    sum_log_likelihood = 0
    for i in range(1, n):
        residual = log_returns[i] - log_returns[i-1] * np.exp(-kappa * dt) - the
```

```

        sum_log_likelihood += t.logpdf(residual, df=nu, loc=0, scale=sigma * np.
        return -sum_log_likelihood

# Initial guess for the parameters
initial_params = [0.0, 0.1, 0.2, 3.0]

# Optimize the parameters using MLE
result = minimize(ou_negative_log_likelihood, initial_params, args=(log_returns,

# Extract the estimated parameters
theta_est, kappa_est, sigma_est, nu_est = result.x

print(f"Estimated Parameters: theta = {theta_est}, kappa = {kappa_est}, sigma =

# Simulate the Process
n_sim = 252
simulated_log_returns = np.zeros(n_sim)
for s in range(1, n_sim):
    simulated_log_returns[s] = simulated_log_returns[s-1] + kappa_est * (theta_e

# Plot the simulated log returns
plt.figure(figsize=(12, 6))
plt.plot(simulated_log_returns, label=f'Simulated {ticker} Log Returns')
plt.title(f'Simulated {ticker} Log Returns using Generalized OU Process with t-d
plt.xlabel('Time (Days)')
plt.ylabel('Log Returns')
plt.legend()
plt.show()

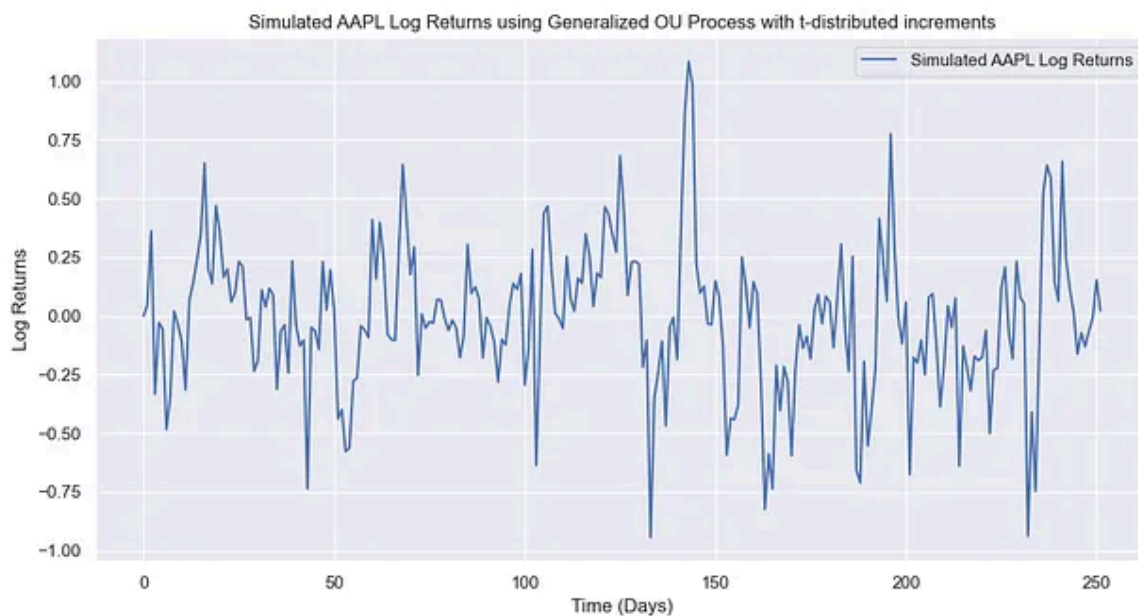
```

Estimated Parameters: theta = 0.006517517871658104, kappa = 0.48678024674719356,

Parameter	Role	Effect
θ	Long-term mean	Determines the level to which the process reverts over time.
κ	Rate of mean reversion	Controls how quickly the process returns to θ .
σ	Volatility (amplitude)	Determines the magnitude of random fluctuations.
ν	Degrees of freedom (optional)	Governs the heaviness of the noise distribution's tails (lower $\nu \rightarrow$ heavier tails).

Optimization algorithms like *scipy.optimize.minimize* are designed to minimize a target function, not maximize it. By minimizing the negative log-likelihood, we achieve the same result as maximizing the original log-likelihood. This ensures compatibility with standard optimization routines.

Logarithms convert products into sums, so the log-likelihood simplifies calculations and avoids underflow.



The Heston Model is a popular stochastic volatility model for pricing financial derivatives, particularly options. It models the underlying asset price using a geometric Brownian motion and the volatility as a mean-reverting square-root process.

To incorporate a t-distribution for the returns, it can modify the simulation of the model to include a t-distribution instead of a normal distribution.

Heston Model

Definition:

The Heston model is used to model stochastic volatility and is described by the coupled SDEs:

$$\begin{aligned}dS_t &= \mu S_t dt + \sqrt{v_t} S_t dW_t^s \\ dv_t &= \kappa(\theta - v_t)dt + \xi \sqrt{v_t} dW_t^v\end{aligned}$$

where:

- S_t : Asset price.
- v_t : Variance (stochastic volatility).
- θ : Long-term variance.
- κ : Speed of mean reversion for variance.
- ξ : Volatility of volatility.
- W_t^s and W_t^v : Correlated Brownian motions (ρ correlation).

Characteristics:

- Variance v_t reverts to θ at rate κ .
- Captures stochastic volatility, enabling better pricing of options and derivatives.
- Allows skewness and kurtosis in asset price distributions.

Where:

$$dW_t^s = \rho dW_t^v + \sqrt{1 - \rho^2} dZ_t$$

Here, dZ_t is a t-distributed random variable.

Discretization

Use Euler-Maruyama or Milstein methods for numerical simulation. In discrete form:

1. Variance:

$$V_{t+\Delta t} = V_t + \kappa(\theta - V_t)\Delta t + \xi\sqrt{V_t\Delta t}\varepsilon_v$$

2. Asset price:

$$S_{t+\Delta t} = S_t \exp \left(\left(\mu - \frac{V_t}{2} \right) \Delta t + \sqrt{V_t\Delta t}\varepsilon_s \right)$$

Where:

$$\varepsilon_v, \varepsilon_s \sim t_\nu$$

Parameter	Represents	Effect on Dynamics	Effect on Option Prices
κ	Speed of mean reversion	Faster/slower variance reversion	Affects stability of implied volatility
θ	Long-term variance	Determines average volatility	Sets baseline implied volatility
ξ	Volatility of variance (vol of vol)	Variance variability	Shapes implied volatility skew/smile
ρ	Correlation between price & variance	Leverage effect	Influences skewness of implied volatility
ν	Degrees of freedom (t-distribution)	Tail heaviness of returns	Captures extreme market moves

Heston Model Simulation:

```
from scipy.optimize import minimize
from scipy.stats import t

# Fetch AAPL stock data
log_returns = aapl['Log Return'].dropna()
```

```

=====
# Estimated Heston Model Parameters
=====

trading_days_per_year = 252

# Initial V0 (Annualized)
V0 = log_returns.var() * trading_days_per_year
# Annualized drift
mu = log_returns.mean() * trading_days_per_year

# Realized variance
realized_variance = log_returns ** 2

# Define Heston variance loss function
def heston_variance_loss(params, realized_variance):
    kappa, theta, xi = params
    V = [V0]
    for i in range(1, len(realized_variance)):
        Vt = V[-1]
        delta_V = kappa * (theta - Vt) + xi * np.sqrt(max(Vt, 0)) * np.random.no
        V.append(Vt + delta_V)
    V = np.array(V)
    return np.sum((realized_variance.values - V) ** 2)

# Optimize kappa, theta, xi
initial_guess = [2.0, 0.02, 0.3]
result = minimize(heston_variance_loss, initial_guess, args=(realized_variance),
kappa, theta, xi = result.x

# Estimate rho
delta_variance = realized_variance.diff().dropna()
rho = np.corrcoef(log_returns[1:], delta_variance)[0, 1]

# Fit t-distribution for nu
params = t.fit(log_returns)
nu = params[0]

# Display results
print("Estimated Parameters:")
print(f"V0 (Initial Variance): {V0:.6f}")
print(f"mu (Drift): {mu:.6f}")
print(f"kappa (Mean Reversion Speed): {kappa:.6f}")
print(f"theta (Long-Term Variance): {theta:.6f}")
print(f"xi (Vol of Vol): {xi:.6f}")
print(f"rho (Correlation): {rho:.6f}")
print(f"nu (Degrees of Freedom): {nu:.6f}")

=====
# Heston Model Simulation

```

```

=====

# Parameters (use the estimated values from earlier)
T = 1.0          # Time horizon in years
dt = 1 / 252     # Daily time steps
N = int(T / dt)  # Number of steps
n_paths = 20     # Number of paths

# Heston parameters
V0 = V0          # Initial variance
S0 = aapl['Adj Close'][-1] # Initial asset price
mu = mu          # Drift
kappa = kappa    # Speed of mean reversion
theta = theta    # Long-term variance
xi = xi         # Volatility of variance
rho = rho        # Correlation
nu = nu         # Degrees of freedom for t-distribution

# Simulate paths
np.random.seed(42)
V = np.zeros((N, n_paths))
S = np.zeros((N, n_paths))
V[0, :] = V0
S[0, :] = S0

# Generate random shocks from t-distribution
for i in range(1, N):
    z1 = t.rvs(df=nu, size=n_paths) # t-distributed random variables for return
    z2 = t.rvs(df=nu, size=n_paths) # t-distributed random variables for variance
    z2 = rho * z1 + np.sqrt(1 - rho ** 2) * z2 # Correlated random variables

    # Variance process
    V[i, :] = V[i - 1, :] + kappa * (theta - V[i - 1, :]) * dt + xi * np.sqrt(np.maximum(V[i - 1, :], 0)) * z1
    V[i, :] = np.maximum(V[i, :], 0) # Ensure variance is non-negative

    # Asset price process
    S[i, :] = S[i - 1, :] * np.exp((mu - 0.5 * V[i - 1, :]) * dt + np.sqrt(V[i - 1, :]) * z2)

# Plot results
time = np.linspace(0, T, N) * 252 # Convert to days

plt.figure(figsize=(14, 6))

# Asset price paths
plt.subplot(1, 2, 1)
plt.plot(time, S)
plt.title(f"Simulated {ticker} Adj Close Price Paths (Heston Model)")
plt.xlabel("Time (Days)")
plt.ylabel("Asset Price")

```



```
# Variance paths
plt.subplot(1, 2, 2)
plt.plot(time, V)
plt.title(f"Simulated {ticker} Variance Paths (Heston Model)")
plt.xlabel("Time (Days)")
plt.ylabel("Variance")

plt.tight_layout()
plt.show()
```

Estimated Parameters:

V_0 (Initial Variance): 0.100779

μ (Drift): 0.246944

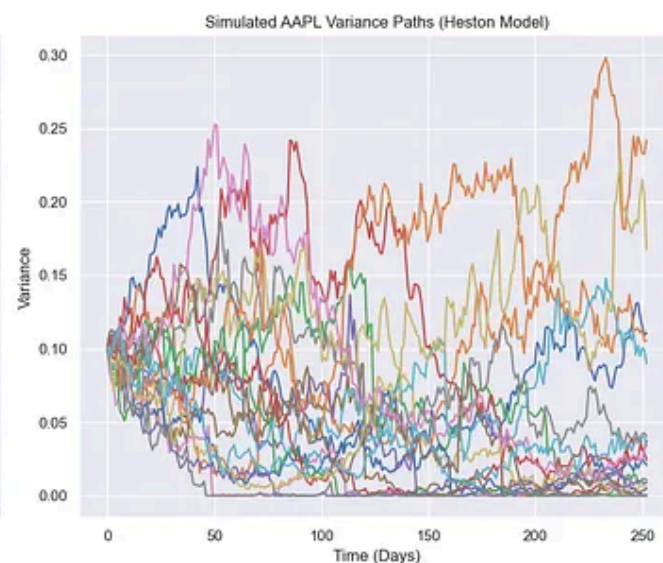
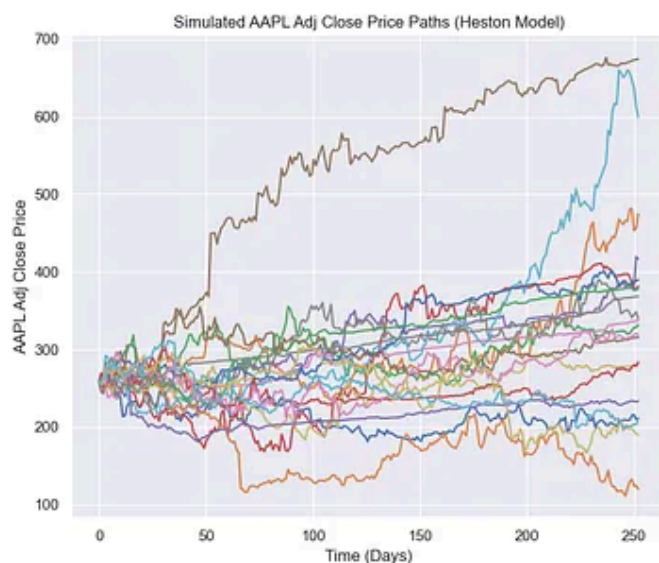
κ (Mean Reversion Speed): 1.170294

θ (Long-Term Variance): 0.024409

ξ (Vol of Vol): 0.324465

ρ (Correlation): 0.001391

ν (Degrees of Freedom): 3.572699



Key differences between OU Process and Heston Model:

Feature	OU Process	Heston Model
Primary Focus	Single-variable mean-reversion	Mean-reverting stochastic volatility
Dimension	1-dimensional	2-dimensional (price and variance)
Volatility	Constant	Stochastic (variance is dynamic)
Applications	Interest rates, energy prices	Equity derivatives, volatility modeling
Distribution	Gaussian	Non-Gaussian (due to stochastic variance)
Complexity	Simple	Moderate to high

Exhibit Volatility Clustering:

Plot the returns and the rolling volatility over time. Volatility clustering is visually evident if periods of high volatility are followed by more high volatility and periods of low volatility are followed by more low volatility.

```
# Define high volatility threshold
high_vol_threshold = aapl['21-Day Rolling Volatility'].quantile(0.8) # Top 20%
aapl['High_Vol'] = aapl['21-Day Rolling Volatility'] > high_vol_threshold

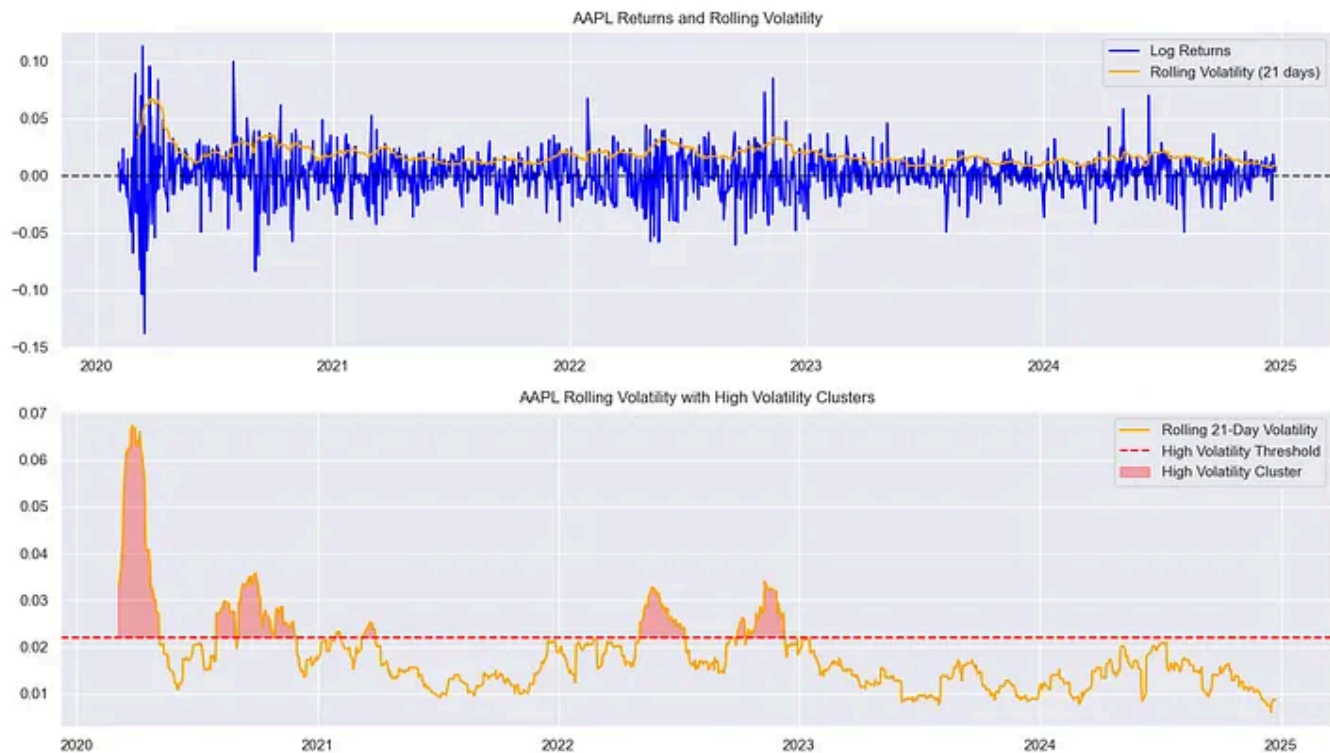
# Plot Returns and Rolling Volatility
plt.figure(figsize=(14, 8))
plt.subplot(2, 1, 1)
plt.plot(aapl['Log Return'].dropna(), label='Log Returns', color='blue')
plt.plot(aapl['21-Day Rolling Volatility'], label='Rolling Volatility (21 days)')
plt.axhline(0, color='black', linestyle='--', alpha=0.7)
plt.title('AAPL Returns and Rolling Volatility')
plt.legend()

# Plot Rolling Volatility with High Volatility Clusters
plt.subplot(2, 1, 2)
plt.plot(aapl['21-Day Rolling Volatility'], label='Rolling 21-Day Volatility', c
plt.axhline(high_vol_threshold, color='red', linestyle='--', label='High Volatil
plt.fill_between(aapl.index, aapl['21-Day Rolling Volatility'], high_vol_thresho
```

```

color='red', alpha=0.3, label='High Volatility Cluster')
plt.title('AAPL Rolling Volatility with High Volatility Clusters')
plt.legend()
plt.tight_layout()
plt.show()

```



```

# Ljung-Box Test
from statsmodels.stats.diagnostic import acorr_ljungbox
lb_test = acorr_ljungbox(aapl['Log Return']**2, lags=50)
print(lb_test)

```

	lb_stat	lb_pvalue
1	134.843981	3.571599e-31
2	248.073541	1.353677e-54
3	281.730852	8.939507e-61
4	348.553564	3.599811e-74
5	412.875938	4.975414e-87
6	485.692573	1.014863e-101

```
.....
45  872.615743  8.208311e-154
46  872.649676  3.580932e-153
47  872.837346  1.436364e-152
48  872.862643  6.156180e-152
49  872.948518  2.536901e-151
50  873.069728  1.017568e-150
```

Interpretation

- All P-Values < 0.05: Since all p-values are less than 0.05, you reject the null hypothesis at each lag. This indicates that there is significant evidence of autocorrelation in the squared returns at all tested lags.
- Volatility Clustering: The presence of autocorrelation in the squared returns suggests that the volatility of the returns is not constant over time but rather exhibits clustering. Periods of high volatility tend to be followed by more high volatility, and periods of low volatility tend to be followed by more low volatility.

Perform the Breusch-Pagan test:

The Breusch-Pagan test is used to determine whether or not heteroscedasticity is present in a regression model.

Null Hypothesis (H0): Homoscedasticity is present (the residuals are distributed with equal variance)

Alternative Hypothesis (HA): Heteroscedasticity is present (the residuals are not distributed with equal variance)

```
import statsmodels.api as sm
import statsmodels.stats.api as sms
```

```
aapl.dropna(subset=['Log Return'], inplace=True)
# Ensure the index is properly reset

aapl.reset_index(inplace=True)

# Linear regression model
X = sm.add_constant(aapl.index) # Explanatory variable
y = aapl['Log Return'] # Dependent variable
model = sm.OLS(y, X).fit()

# Breusch-Pagan test
bp_test = sms.het_breuschpagan(model.resid, model.model.exog)
print(f"Breusch-Pagan test statistic: {bp_test[0]}, p-value: {bp_test[1]}")
```

Breusch-Pagan test statistic: 48.010647526624325, p-value: 4.2391086628921855e-1

Interpretation:

- **Test Statistic:** A higher value may indicate the presence of heteroskedasticity.
- **p-value:** A low p-value (e.g., less than 0.05) suggests rejecting the null hypothesis of homoskedasticity, indicating that heteroskedasticity is present.

Key Differences

Aspect	Autocorrelation	Heteroskedasticity
Focus	Dependency between time series values.	Changes in the variance over time.
Metric	Correlation coefficient at different lags.	Variance (and its clustering).
Measurement	Applied to the levels, squared, or absolute returns.	Related directly to the variance of residuals.
Volatility Clustering	Observed in squared or absolute returns.	Key characteristic, creating periods of high and low volatility.

Generalized Autoregressive Conditional Heteroskedasticity:

GJR-GARCH (*Glosten-Jagannathan-Runkle GARCH*) models are widely used in finance to model and forecast time-varying volatility. Useful in financial markets where negative shocks (e.g., bad news) result in disproportionately higher volatility compared to positive shocks (e.g., good news).

GJR-GARCH: Extends GARCH to include an asymmetry term to capture the leverage effect (larger impact of negative shocks on volatility).

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \gamma \epsilon_{t-1}^2 I(\epsilon_{t-1} < 0) + \beta \sigma_{t-1}^2$$

where:

- σ_t^2 is the conditional variance at time t ,
- $\omega > 0$, $\alpha \geq 0$, and $\beta \geq 0$ are model parameters,
- ϵ_{t-1} is the past residual (innovation),
- α captures the impact of past shocks,
- β captures the persistence of volatility,
- $I(\epsilon_{t-1} < 0)$ is an indicator function that equals 1 if $\epsilon_{t-1} < 0$, otherwise 0,
- γ captures the additional effect of negative shocks (leverage effect).

Install required library:

```
# https://arch.readthedocs.io/en/latest/index.html
pip install arch
```

```
from arch import arch_model

# Define the range of values for p, o, and q
p_values = range(1, 2) # Reduced range for p
o_values = range(0, 2)
q_values = range(1, 2) # Reduced range for q

best_aic = float('inf')
best_order = None
best_model = None

# Perform grid search
for p in p_values:
    for o in o_values:
        for q in q_values:
```

```

try:
    model = arch_model(aapl['Log Return'], vol='Garch', p=p, o=o, q=q)
    model_fit = model.fit(disp='off', options={'maxiter': 1000})
    if model_fit.aic < best_aic:
        best_aic = model_fit.aic
        best_order = (p, o, q)
        best_model = model_fit
except Exception as e:
    print(f"Error fitting model with (p, o, q) = ({p}, {o}, {q}): {e}")
    continue

# Display the best model parameters
print(f"Best order (p, o, q): {best_order}")
print(f"Best AIC: {best_aic}")
print(best_model.summary())

# Forecast future volatility using the best model
forecasts = best_model.forecast(horizon=10)

# Display the forecasted volatility
print(forecasts.variance.iloc[-1:])

```

Best order (p, o, q): (1, 1, 1)

Best AIC: -6502.81815404072

Constant Mean - GJR-GARCH Model Results

```

=====
Dep. Variable:          Log Return      R-squared:          0
Mean Model:             Constant Mean   Adj. R-squared:      0
Vol Model:              GJR-GARCH       Log-Likelihood:     325
Distribution:           Standardized Student's t    AIC:               -650
Method:                Maximum Likelihood    BIC:               -647
                                           No. Observations:
Date:                  Wed, Dec 25 2024    Df Residuals:
Time:                  11:24:29           Df Model:
                                           Mean Model
=====

```

	coef	std err	t	P> t	95.0% Conf. Int.
mu	1.1311e-03	4.598e-04	2.460	1.390e-02	[2.299e-04, 2.032e-03]

Volatility Model

	coef	std err	t	P> t	95.0% Conf. Int.
omega	7.9952e-06	3.195e-08	250.235	0.000	[7.933e-06, 8.058e-06]
alpha[1]	0.0500	2.687e-02	1.861	6.274e-02	[-2.658e-03, 0.103]

```
gamma[1]      0.0500  3.073e-02    1.627    0.104  [-1.023e-02,  0.110]
beta[1]       0.9050  7.259e-03   124.668    0.000    [  0.891,  0.919]
```

Distribution

```
=====
              coef      std err          t      P>|t|   95.0% Conf. Int.
-----
nu              6.9043      4.539      1.521    0.128  [ -1.992, 15.801]
=====
```

Covariance estimator: robust

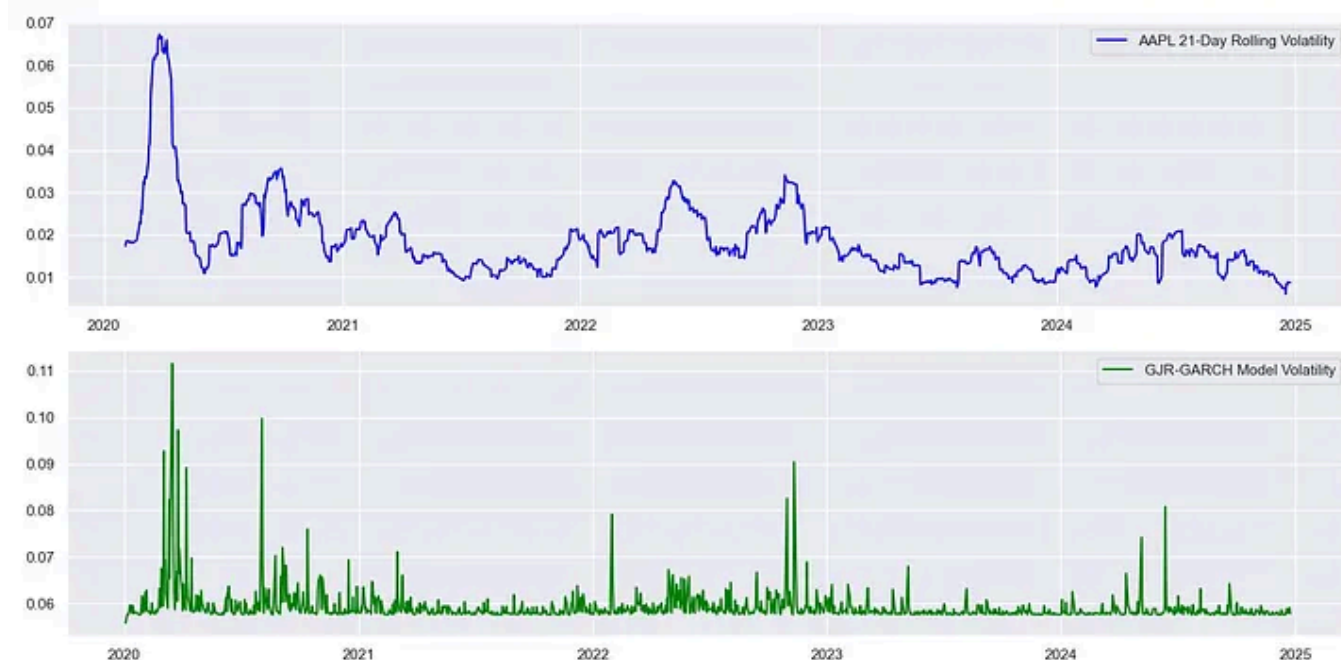
```
              h.01      h.02      h.03      h.04      h.05      h.06  \
Date
2024-12-24  0.000163  0.000168  0.000173  0.000177  0.000182  0.000186
```

```
              h.07      h.08      h.09      h.10
Date
2024-12-24  0.00019  0.000195  0.000199  0.000203
```

```
# Plot the results
plt.figure(figsize=(14, 7))

# Plot Historical Volatility
plt.subplot(2, 1, 1)
plt.plot(aapl['21-Day Rolling Volatility'], label=f"{ticker} 21-Day Rolling Vola
plt.legend()

# Plot GARCH Model Volatility
plt.subplot(2, 1, 2)
plt.plot(best_model.conditional_volatility, label="GJR-GARCH Model Volatility",
plt.legend()
plt.tight_layout()
plt.show()
```

Overall Interpretation:

The best order $(p,o,q)=(1,1,1)$ for the GJR-GARCH model suggests a parsimonious model that captures both the persistence of volatility and the asymmetric response to returns. The model parameters provide insights into the dynamics of volatility, including the leverage effect and the impact of past returns on current volatility.

- Mean Return (μ): The model suggests a small but significant positive average return.
- Baseline Volatility (ω): The baseline level of volatility is very low but highly significant.
- ARCH Effect ($\alpha[1]$): Past squared returns have a significant impact on current volatility.
- Asymmetric Effect ($\gamma[1]$): The leverage effect is not significant in this model, suggesting that negative returns do not have a different impact on volatility compared to positive returns.

- Volatility Persistence ($\beta[1]$): Volatility is highly persistent, with a significant coefficient for the lagged variance term. This indicates that volatility tends to cluster over time.
- Distribution (ν): The use of the Student's t-distribution with a significant degrees of freedom parameter captures the heavy tails in the return distribution, making the model more robust to extreme values.

Thank you for reading. [[Full source code here](#)]

Other articles of GARCH, Heston Model, and Ornstein-Uhlenbeck Process:

[How to Predict Stock Volatility Using GARCH Model In Python](#)

[How to Predict Stock Volatility with Python](#)

[Volatility Analysis and Forecasting for AAPL Stock Using GARCH Model](#)

[Building a GARCH Volatility Model in Python: A Step-by-Step Tutorial with Statistical Analysis](#)

[Forecasting Volatility: Deep Dive into ARCH & GARCH Models](#)

[S&P500 Volatility: ARCH vs GARCH Models](#)

[Overcoming Chaos: Forecasting Volatility Models Using Python](#)

[Simulating the Heston model with the Quadratic Exponential scheme](#)

[Stochastic Processes Simulation — The Ornstein Uhlenbeck Process](#)

[How to Properly Model Asset Volatility with Python Using the Ornstein-Uhlenbeck Model](#)

Disclaimer:

The information presented herein is provided solely for informational purposes and should not be construed as financial or investment advice. Any investment decisions or actions taken based on this information are made at your own discretion and risk. Stock prices and investment values are subject to fluctuation due to various market factors and conditions.

I strongly recommend consulting with a qualified financial professional for personalized guidance tailored to your individual financial situation and objectives before making any investment decisions.

**Written by Simon Leung**

229 followers · 4 following

[Edit profile](#)

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.

